TranKhanhBang_REPORT

Lab 3: Individual Report

Student Name: [Your Name]
Student ID: [Your ID]

I. Technical Contribution

List the specific code modules or scripts you implemented. Be specific.

- Example: Implemented the `compare_chatbot_vs_agent.py` script.
- Example: Fixed the `hard_solver.py` timeout loop issue.

I did the following jobs:

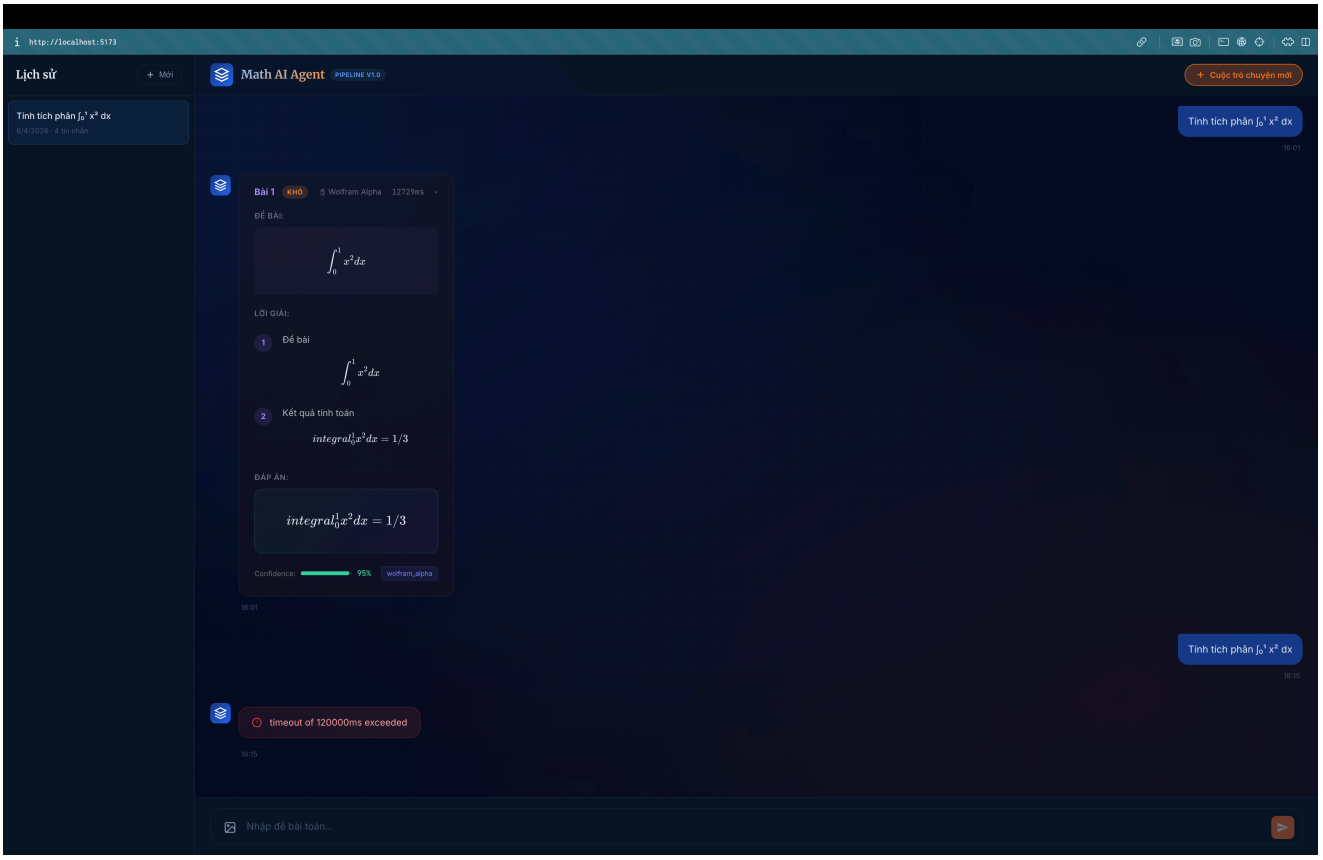
- Analyze the seletons and give the initial scaffold for our team
- implement the app/api
- Responsible for review and merge teammate's code
- Ensure the final code is bug-free and run perfectly

II. Debugging Case Study

Provide a detailed analysis of one failure you personally helped debug. It could be a hallucination, an infinite loop, or a Python syntax parser error.

The Error:
The origin UI have some problem with clicks
I cant click new questions, and when one question is submitted, It can not receive new ones
The backend log can handles edges cases but the frontend cant display them.

The Logs:



The Fix:

New question: There is a attribute record the state of input to prevent user input another questions, however this attribute is not refresh when answers sucess

New chatbutton: It is hide from the logo so I move the Math AI Agent to the right

FE not displaying edges cases: The bug was fixed by implementing a "Fail-Conversational" routing in the backend and hardening the frontend state management. On the backend, we modified the LangGraph `routing.py` and `graph.py` to route "No Math Found" cases to the `aggregator` rather than an error node, allowing the agent to reply conversationally. On the frontend, we updated `ChatInput.tsx` and `ChatWindow.tsx` with mandatory `finally` blocks to reset the `isLoading` state, and modified `ChatMessage.tsx` to render text content (`message.content`) even when the mathematical `results` array is empty. Suggestion buttons were also disabled during loading to prevent state corruption via concurrent clicks

III. Personal Insights

Reflect on the fundamental differences between LLM Chatbots vs ReAct Agents based on your experience coding them.

In terms of coding experiences, I have never code a LLM chatbots ever before, I only call api and use them directly. For ReAct Agents, however, I have a good time coding them today, involves using tools, implements features, work with teammates. I learnt a lot more while coding them, compares to LLM chatbots.

In terms of fundamentals, chatbot only give suggestion, fix code, ... or in other words, give text outputs. For ReAct Agents, they have more power, actions. They call the nescessary tools and run them, feeding the results again automatically, making them perform much more faster and efficient

IV. Future Improvements

Propose an idea for scaling this system to a production-level standard. What would you add next? (e.g., RAG memory, human-in-the-loop, parallel tool calling).

To make this system into a production level, I think a parallel tool calling feature to improve latency and human-in-the-loop to handles fallbacks. Right now, the pipeline primarily operates sequentially. However, complex mathematical queries could be processed much faster if the planner node asynchronously dispatched multiple tools at once (e.g., executing Python equations while simultaneously querying Wolfram Alpha) and the aggregator simply chose the fastest, most reliable output.

Additionally, we should integrate a HITL layer: if the agent enters a persistent failure loop or encounters a problem it cannot solve after maximum retries, the system should pause the agent and dynamically escalate the chat to a live support tutor. Lastly, adding a **RAG (Retrieval-Augmented Generation) memory cache** to index previously solved, complex mathematical formulas would drastically cut LLM token costs and lower latency for frequently asked standard curriculum questions.